Coaching Qwen3 Coder 30B to Think Like a CodeClash Arena Agent

Stanford CS224N Custom Project

Ivy (Ning) Zhang
Department of Computer Science
Stanford University
para2046@stanford.edu

Abstract

Large language model coding agents have recently become useful for software tasks, but weaker or open-weight agents still struggle to reliably interpret user intent and execute complex multi-step workflows. This gap is especially visible in long-horizon settings, where an agent must repeatedly inspect prior outcomes, diagnose failure, and choose the next code edit under interaction constraints. It motivates a natural question: what can we do to improve a weak code agent's thinking process? We study this question in CodeClash, a code-arena benchmark where the original work evaluates 8 commercial coding agents across 6 arenas through multi-round tournaments. Since Qwen3 Coder Plus ranks last among them, we take the open-weight Qwen3-Coder-30B as a case study and investigate how to improve it with distilled knowledge from stronger agents. Our analysis shows that Qwen3-Coder-30B is not well optimized for arena-style interaction: it frequently produces syntax and protocol-breaking errors and exhibits weak strategic adaptation across rounds. These failures are difficult to correct with vanilla instruction tuning alone, since offline SFT cannot directly verify whether a generated action is valid or beneficial. To address this, we propose ReAct SFT, which rewrites teacher trajectories into explicit `[obs][thought][act]` chains, and trajectory-quality weighted SFT, which reweights samples to encourage post-edit checking. ReAct SFT substantially improves strategic behavior, and our fine-tuned model outperforms the original Qwen3 Coder Plus in tournament evaluation.

Mentor. We thank our mentor Chenglei Si for his guidance throughout the project and for helping us obtain the original tournament trajectories collected by CodeClash author John Yang, which made these experiments possible.

Late Days. We used 1 late day for this report.

1 Introduction

In practical coding workflows, we observe a clear gap between weaker and stronger coding agents. Weaker or free tier systems often fail to properly interpret user instructions, lose track of complex requests, or behave unreliably over multi step interactions, whereas stronger frontier agents can handle the same tasks much more consistently. This gap is not fully explained by standard coding benchmarks, which mainly emphasize one shot code generation or issue resolution, such as HumanEval style generation tasks [1] and repository level bug fixing benchmarks like SWE bench [2]. These observations motivate our central question: how can we improve the long horizon behavior of a weak coding agent?

We study this question in CODECLASH [3], a benchmark platform for goal oriented, long horizon software engineering. In CODECLASH, each model iteratively edits a private repository and then

competes with other edited codebases in a code arena, i.e., an executable environment where repository quality is measured through downstream head to head competition rather than direct task completion. The original benchmark evaluates 8 frontier coding agents—Claude Sonnet 4.5, GPT 5, o3, Claude Sonnet 4, GPT 5 Mini, Gemini 2.5 Pro, Grok Code Fast, and Qwen3 Coder—across 6 code arenas through multi round tournaments [3]. Among them, Claude Sonnet 4.5 ranks first overall, while Qwen3 Coder ranks last. This motivates our use of the open weight Qwen3 Coder 30B as a posttraining case study, and we use the recorded Claude Sonnet 4.5 trajectories as the teacher source for ReAct SFT supervision. In our experiments, Qwen3 Coder 30B frequently produces syntax and protocol breaking errors and under uses behaviors encouraged by the arena, such as log inspection, outcome analysis, and post edit validation.

A key challenge is that vanilla instruction tuning provides only weak action level control: offline supervised finetuning does not directly tell the model whether a generated action is protocol legal or strategically useful for future rounds. To address this, we build on existing CODECLASH trajectories and introduce two forms of supervision. ReAct SFT rewrites strong agent trajectories into explicit `[Obs][Thought][Act]` annotations while preserving the original bash command, making the competitive decision process more explicit. Trajectory Quality Weighted SFT (TQ-SFT) instead adds a reliability oriented control signal by upweighting trajectories that follow safer modify then check behavior. Empirically, ReAct SFT yields the strongest gains in competitive behavior, while TQ-SFT mainly reduces invalid submissions without producing equally strong tournament improvements.

2 Related Work

2.1 Code arenas and interactive software engineering benchmarks

Most standard coding benchmarks evaluate models on well specified local tasks, such as code generation from docstrings or issue resolution in a fixed repository context. For example, HumanEval measures functional correctness for one shot code generation [1], while SWE bench evaluates whether models can resolve real GitHub issues by producing patches that pass repository tests [2]. Interactive coding benchmarks such as InterCode further emphasize execution feedback during problem solving [4]. In contrast, CODECLASH frames coding as goal oriented, long horizon software engineering: agents iteratively edit private repositories and compete in multi round tournaments across executable code arenas [3]. This makes CODECLASH a natural benchmark for our work, since our goal is not only to improve local code generation, but to improve agent behavior under repeated feedback and competitive pressure.

2.2 Reasoning and acting supervision for language agents

Our ReAct style supervision is motivated by prior work on language agents that interleave reasoning and action. ReAct shows that explicitly structuring model outputs as reasoning traces followed by task specific actions can improve sequential decision making and robustness in interactive settings [5]. More broadly, recent language agent overviews argue that effective agents require explicit reasoning, grounding in feedback, and action selection under changing context [6]. These ideas are directly relevant to our setting: in CODECLASH, an agent must observe logs and previous outcomes, reason about why a round was won or lost, and then choose the next edit. Our ReAct SFT adapts this perspective to code arenas by rewriting teacher trajectories into explicit `[Obs][Thought][Act]` supervision.

2.3 Instruction tuning and its limits in long horizon coding

Instruction tuning is known to substantially improve model usability and generalization across tasks [7]. However, open resource studies also show that instruction tuned models acquire different capabilities depending on the supervision data, and that no single instruction tuning recipe uniformly transfers all skills [8]. This is particularly relevant in our setting. Our preliminary experiments suggest that vanilla instruction tuning over arena trajectories can teach Qwen3 Coder 30B to imitate the surface format of agent responses, but not necessarily the deeper observation reasoning action loop required for strong arena play. This motivates our comparison between two more targeted posttraining signals: ReAct style structured supervision, which aims to transfer competitive reasoning, and trajectory quality weighting, which aims to improve reliability under the arena protocol.

3 Task Setting

We study weak agent improvement in the CODECLASH BattleSnake arena [3]. In CODECLASH, coding agents iteratively modify a private repository and are evaluated through multi round tournaments. Each round alternates between an edit phase, where the agent interacts with a terminal scaffold under a fixed budget, and a competition phase, where the resulting codebase is executed in the arena. Competition outputs, including logs and round results, are copied back into the repository and become the main source of feedback for the next round. In the BattleSnake setting, the agent must follow a strict single action protocol: at each step it produces exactly one bash command together with a THOUGHT section, receives the execution result, and then continues to the next step.

Our focus is therefore not only whether Qwen3 Coder 30B can generate plausible code, but whether it can behave like an arena agent under repeated feedback. In preliminary experiments, we observe two recurring weaknesses: execution fragility, including syntax and protocol failures, and weak strategic adaptation, including limited use of logs, little post edit validation, and shallow use of previous outcomes. Our goal is to improve both reliability and competitive reasoning in this long horizon setting.

4 Approach

Our posttraining pipeline begins with standard supervised finetuning baselines, including self play SFT and teacher distilled SFT, and then introduces two more targeted supervision signals: ReAct SFT, which makes the observation reasoning action structure explicit, and Trajectory Quality Weighted SFT (TQ-SFT), which reweights training examples toward safer modify then check behavior.

4.1 Shared SFT Setup

Following prior instruction tuning practice [8], we treat each CODECLASH round trajectory as a multi turn conversational finetuning instance. Each trajectory consists of one system message s and a sequence of user–assistant exchanges. For the k-th assistant response a_k, we define the history context as the prefix up to the current user turn:

$$X_k = (s,\ u_1,\ a_1,\ \ldots,\ u_{k-1},\ a_{k-1},\ u_k),$$

where u_k denotes the current user-side observation and a_k is the corresponding assistant output, e.g., a THOUGHT plus one bash action block.

We serialize the full trajectory into a token sequence $t_{1:L} = (t_1, \ldots, t_L)$. Let Y denote the set of token positions that belong to assistant spans. We then optimize an assistant only supervised finetuning objective:

$$\mathcal{L}_{\mathrm{SFT}} = -\sum_{j=1}^{L} \log p_\theta(t_j \mid t_{<j}) \cdot \begin{cases} 1, & j \in Y, \\ 0, & \text{otherwise.} \end{cases}$$

That is, we compute loss only on assistant tokens and mask out system and user tokens. This choice is important because it directly trains the model on the agent outputs we want to improve, rather than on the full conversation transcript.

To enable efficient adaptation of a 30B decoder only model under long context training, we use QLoRA/LoRA style parameter efficient finetuning [9]. In our implementation, the backbone model is loaded in 4 bit quantized form, while forward and backward computation are performed in bfloat16 precision. LoRA adapters with rank $r = 16$ are inserted into the attention projection layers (`q_proj`, `k_proj`, `v_proj`, and `o_proj`) of all Transformer blocks, while the original backbone weights remain frozen. We also support both full trajectory training and fixed size windowing over assistant turns, since tournament trajectories can be long while the most relevant dependencies are often local to the recent interaction history.

4.2 ReAct SFT

Our first method addresses a limitation of vanilla instruction tuning: the model can learn to imitate the surface form of an arena response—for example, a plausible THOUGHT followed by a bash

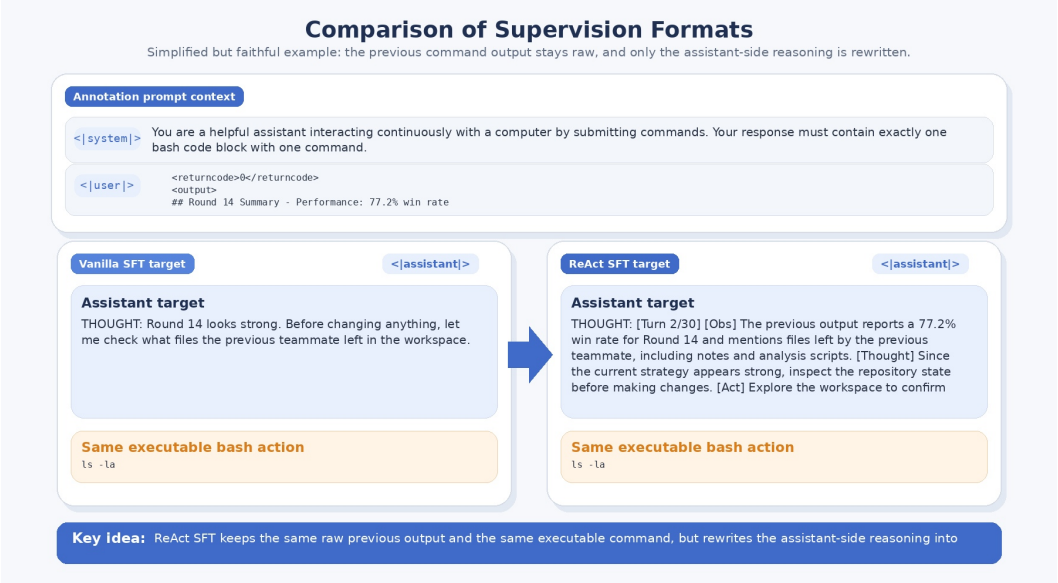

Figure 1: A simplified but faithful example illustrating our supervision rewriting scheme. The raw previous output is preserved in context, but the assistant-side reasoning target is rewritten from an unstructured `THOUGHT` into explicit `[Obs]` `[Thought]` `[Act]` fields, while preserving the original executable bash action.

command—without learning how to reason strategically under feedback. To make the intended decision process more explicit, we construct ReAct SFT supervision by rewriting teacher trajectories into structured `[Obs]` `[Thought]` `[Act]` targets. Figure 1 illustrates the difference between vanilla instruction style supervision and our ReAct style reformulation.

Concretely, we use Anthropic's Claude Sonnet 4.5 API to annotate strong agent trajectories step by step; in this setting, the model rewrites its own play trajectories. For each assistant turn, the annotation prompt includes the previous execution output, the current step context, the original THOUGHT text, and the original bash command. The annotator then rewrites only the reasoning portion into three parts: `[Obs]`, which summarizes the most relevant previous feedback; `[Thought]`, which states the strategic purpose of the current step; and `[Act]`, which names the concrete operation being executed. The original bash command is preserved, so the rewritten trajectory remains faithful to the teacher action while making the observation reasoning action structure explicit.

4.3 Trajectory Quality Weighted SFT (TQ-SFT)

Our second method targets a different failure mode: invalid or poorly validated edits. In preliminary experiments, Qwen3 Coder 30B frequently makes large modifications without checking whether the code still runs, occasionally removes critical scaffold structure, and submits broken code. This suggests that a useful auxiliary signal is not only what action was taken, but whether the trajectory reflects a more reliable editing process.

To capture this, we construct rule derived trajectory annotations from the top 5 teacher trajectories. Assistant bash commands are mapped into coarse behavior categories such as `MODIFY_MAIN`, `CODE_CHECK`, `SUBMIT`, `STRATEGY_READ`, and `STRATEGY_ANALYSIS`. From the temporal order of these actions, we compute trajectory level statistics such as how often a code modification is followed by checking before submission, and how often the agent submits without validating its changes. These statistics are combined into a heuristic trajectory quality score $q_i \in [0, 1]$, designed to reward modify then check behavior and penalize blind modifications.

During training, this score is converted into a per sample weight w_i, and the supervised loss is reweighted accordingly:

$$L_{\text{TQ-SFT}} = \frac{\sum_i w_i \, \ell_i}{\sum_i w_i},$$

where ℓ_i is the assistant only cross entropy loss for sample i. Intuitively, trajectories that follow a safer edit validate submit pattern contribute more strongly to optimization, while trajectories that modify code without checking are downweighted. Unlike ReAct SFT, TQ-SFT does not explicitly teach the model how to reason about competition; rather, it biases the model toward more reliable arena behavior.

5 Experiments

We evaluate whether posttraining can improve the long horizon arena behavior of Qwen3 Coder 30B in CODECLASH BattleSnake. Our experiments compare three forms of posttraining: vanilla self play SFT, ReAct SFT, and Trajectory Quality Weighted SFT (TQ-SFT). Unless otherwise noted, all evaluations are conducted in the same BattleSnake scaffold and follow the original tournament setup of CODECLASH [3].

5.1 Experimental Setup

All posttraining variants start from the same open weight base model, `Qwen3 Coder 30B A3B Instruct`, which we refer to as Qwen3 Coder 30B. We compare the following systems:

- Vanilla self play SFT: supervised finetuning on default arena trajectories.
- ReAct SFT: supervised finetuning on trajectories rewritten into `[Obs][Thought][Act]` format.
- TQ-SFT: supervised finetuning with trajectory quality weighted loss.

All finetuned variants use the same shared SFT framework described in Section 4.

5.2 Training Data

For Vanilla self play SFT, we use arena trajectories in their original format, where the assistant produces an unstructured `THOUGHT` followed by a bash command. We consider three data sources in this setting. First, we collect Qwen self play data by running `Qwen3 Coder 30B` against itself for 10 tournaments, of which roughly 40% of the resulting round trajectories are valid submissions. Second, we use recorded `Gemini 2.5 Pro` teacher trajectories, which contribute 1,005 round level trajectories. Third, we use recorded `Qwen3 Coder Plus` teacher trajectories, which contribute 975 round level trajectories. This setting tests whether standard supervised imitation over arena trajectories, without additional structure, is already sufficient to teach stronger arena behavior.

For ReAct SFT, we construct training data from stronger teacher trajectories by rewriting each assistant step into structured `[Obs][Thought][Act]` supervision while preserving the original bash command. Our main teacher source is the recorded Claude Sonnet 4.5 BattleSnake corpus. We use Claude Sonnet 4.5 itself as the annotator for step level rewriting. In the initial ablations over context length, we use an early rewritten subset of about 150 trajectories. After identifying the most promising setting, we expand the rewritten corpus to up to 430 trajectories and use the larger set to train the final 6 turn model.

For TQ-SFT, we build a teacher dataset from the strongest recorded agents, including Claude Sonnet 4.5, GPT 5, GPT 5 Mini, Claude Sonnet 4, and Gemini 2.5 Pro. Together, these five agents contribute 4,905 completed round level BattleSnake trajectories. We annotate each trajectory with rule derived action labels and trajectory quality scores computed from edit/check/submit patterns, and convert these scores into per sample loss weights during training. The purpose of this variant is to encourage safer modify then check behavior rather than directly teaching stronger competitive strategy.

Player A	Player B	Winner	Win	Lose	Tie	Primary failure side
Qwen3-30B SFT (self-play)	Qwen3-Coder-30B base	Player A	8	2	0	both
Qwen3-30B SFT (qwen-coder-plus)	Qwen3-Coder-30B base	Player B	1	4	0	mostly A
Qwen3-30B SFT (gemini-2.5-pro)	Qwen3-Coder-30B base	Player A	2	1	0	both

Table 1: Preliminary baseline comparisons at the round level, excluding abnormal rounds with outcomes $(0, 0, 1000)$, $(0, 1000, 0)$, or $(1000, 0, 0)$. Winner is determined by the higher number of round wins among valid rounds. Primary failure side indicates whether failed rounds were mainly caused by Player A, Player B, or both sides producing invalid submissions or structural runtime errors.

5.3 Evaluation

We use the CODECLASH framework to benchmark model performance in the BattleSnake arena. Performance is measured at both the round and tournament levels. At the round level, we report non tie win rate across 1,000 simulator runs with different random initializations. At the tournament level, we compare cumulative outcomes across all 15 rounds, including total score, round wins, and tie rounds. In addition to competitive metrics, we also track reliability oriented outcomes such as invalid submission rounds and round collapse behavior.

5.4 Experimental Details

All experiments use QLoRA as described in Section 4. We set the learning rate to 1×10^{-5}, use batch size 1 with gradient accumulation of 8, and compute loss only on assistant tokens with sample level normalization so that longer trajectories do not dominate training. Because CODECLASH trajectories are long and GPU memory is limited, we train on sliding windows of K assistant turns together with their corresponding user observations.

For ReAct SFT, we experiment with context windows of $K \in \{2, 4, 6, 8\}$ in order to identify the most effective context length. For TQ-SFT, we use $K = 6$. We keep the optimization setup fixed across variants so that differences are driven primarily by supervision structure and data quality rather than by training hyperparameters.

5.5 Preliminary Baselines

Before introducing ReAct SFT and TQ-SFT, we ran a set of preliminary SFT experiments using self play data and several teacher sources. Table 1 summarizes these comparisons and reveals a consistent instability in ordinary arena trajectory finetuning. Although vanilla SFT can yield a modest win rate improvement when trained on Qwen's own self play data, it does not reliably teach the deeper interaction loop required for competitive adaptation. Instead, the resulting models remain vulnerable to the structural failures discussed above, including invalid submissions caused by syntax corruption and removal of the BattleSnake starter block. This instability becomes more severe when the student is trained directly on other agents' trajectories: teacher data SFT using `Qwen3 Coder Plus` or `Gemini 2.5 Pro` frequently collapses into all tie rounds or invalid execution states, suggesting that naive imitation can amplify behavior drift rather than transfer robust strategy. See Appendix Section A.1 for a detailed error analysis.

We observe a similar pattern in a self play variant of TQ-SFT; this result is discussed in Section 6.1. Together, these results suggest that vanilla style SFT may help the model adapt to the surface form of its own trajectories, but does not reliably improve strategy and can even increase failure risk at inference time.

6 Results and Analysis

6.1 Main Tournament Results

We first explored ReAct style supervision with context windows of 2, 4, and 8 turns using an initial rewritten subset of about 150 trajectories. Table 2 shows that the 4 turn variant performed best among

Player A	Player B	Winner	Win	Lose	Tie	Primary failure side
ReAct 2 turn	Qwen3 Coder 30B base	Player B	1	12	0	both
ReAct 8 turn	Qwen3 Coder 30B base	Player B	1	12	1	both
ReAct 4 turn	Qwen3 Coder 30B base	Player A	6	2	1	both
TQ-SFT	Qwen3 Coder 30B base	Player B	3	6	3	mostly B
ReAct 6 turn	TQ-SFT	Player A	11	1	2	both
TQ-SFT	Vanilla SFT (self play)	Player A	3	1	0	mostly B
ReAct 4 turn	TQ-SFT	Player A	8	3	3	both

Table 2: Ablation results across post-training variants in BattleSnake, excluding abnormal rounds with outcomes $(0, 0, 1000)$, $(0, 1000, 0)$, or $(1000, 0, 0)$ from the Win/Lose/Tie counts. Winner is determined by the higher number of round wins among valid rounds. 'Primary failure side' summarizes which side mainly caused the excluded abnormal rounds.

Player A	Player B	Winner	Win	Lose	Tie	Primary failure side
ReAct 6 turn	Qwen3 Coder 30B base	Player A	9	3	1	both
ReAct 6 turn	Qwen3 Coder Plus	Player A	12	0	0	mostly B
ReAct 6 turn	Claude 4.5	Player B	0	15	0	none

Table 3: Main tournament comparisons for the strongest model, ReAct 6-turn, excluding abnormal rounds with outcomes $(0, 0, 1000)$, $(0, 1000, 0)$, or $(1000, 0, 0)$ from the Win/Lose/Tie counts. Winner is determined by the higher number of round wins among valid rounds.

these early ablations. ReAct 2 turn appears too short to preserve enough arena history for strategic adaptation, while ReAct 8 turn appears too long and noisy, making it harder for the student to extract stable decision patterns.

By contrast, TQ-SFT is better understood as a reliability oriented baseline than a strong strategic one. It improves some robustness related behaviors, but it still underperforms ReAct 4 turn in direct competition. In a self play variant, TQ-SFT reduced invalid submissions in the generated data from roughly 40% to 25%, yet the resulting checkpoint failed more often in actual tournament play. This suggests that imitation based finetuning alone does not reliably improve strategic behavior and may even increase deployment time failures through behavioral drift.

Motivated by the strong performance of ReAct 4 turn, we then expanded the rewritten corpus to up to 430 trajectories and trained a 6 turn ReAct variant as our final model. Table 3 reports the resulting main tournament comparisons. ReAct 6 turn wins decisively against both `Qwen3 Coder 30B base` and `Qwen3 Coder Plus`, while still remaining clearly below Claude 4.5. Compared with the earlier ReAct variants and TQ-SFT, the final model also shows fewer abnormal collapse rounds, further supporting the value of structured observation reasoning action supervision for competitive code editing agents.

6.2 Discussion

ReAct SFT improves competitive performance primarily by changing the interaction process rather than only the output format. The ReAct trained model is more likely to revisit round results, inspect logs, and choose edits tied to observed failure modes. This is consistent with the gains of the 4 turn and 6 turn variants and suggests that structured supervision transfers part of the observation reasoning action loop used by stronger agents.

TQ-SFT improves a different dimension. By upweighting trajectories with safer modify then check behavior, it reduces invalid and protocol breaking actions and raises the floor of performance. However, it does not directly teach the model which edits are strategically useful, so the model often becomes more cautious without becoming much more adaptive. Table 4 provides a partial behavioral breakdown consistent with this interpretation: ReAct SFT shifts behavior toward round aware diagnosis and testing, whereas TQ-SFT mainly increases checking and analysis behaviors associated with reliability. Detailed behavioral analysis of the tournament runs is provided in Appendix Section A.1.

Action category	Base Qwen	Qwen Coder Plus	Claude 4.5	ReAct SFT	TQ-SFT
Round / result checks (A)	1.22%	4.99%	31.75%	21.51%	9.93%
Edits-log checks (B)	0.00%	0.07%	1.67%	6.69%	0.00%
Simulation slice checks (C)	0.00%	0.40%	18.28%	0.00%	0.00%
Code structure checks (D)	6.94%	14.88%	8.08%	1.16%	0.97%
Local testing (E)	4.90%	0.35%	7.01%	11.34%	8.72%
Analysis scripts (F)	0.00%	0.29%	11.98%	9.88%	12.59%

Table 4: Partial behavioral action breakdown across representative agents in BattleSnake. Percentages denote the share of assistant bash actions assigned to the selected categories shown here, so columns do not sum to 100%. ReAct SFT shifts behavior toward round-aware diagnosis and testing, moving closer to Claude 4.5, while TQ-SFT mainly increases testing and scripted analysis associated with reliability.

7 Conclusion and Future Work

We studied whether posttraining can improve a weaker open weight coding agent in a long horizon competitive coding benchmark. Using `Qwen3 Coder 30B` in CODECLASH BattleSnake, we found that the base model is limited by both execution fragility and weak arena specific reasoning over logs, prior outcomes, and iterative feedback. Across the posttraining variants we evaluated, ReAct style supervision was the most effective: the final 6 turn model, trained after expanding the rewritten corpus beyond the initial ablation subset, substantially outperformed the base model and clearly exceeded TQ-SFT in tournament play.

Overall, our results suggest that improving coding agents for sequential software tasks requires more than standard instruction tuning or reliability oriented filtering. What matters is transferring a feedback driven action policy that helps the model connect observations, reasoning, and edits over repeated rounds. Structured posttraining is therefore a promising direction for open weight coding agents, although substantial robustness gaps still remain. As future work, it would be valuable to compare other open weight models of similar scale and to study stronger posttraining signals that combine strategic reasoning with explicit validation.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023.

[3] John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering, 2025.

[4] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: Standardizing and benchmarking interactive coding with execution feedback. In *NeurIPS Datasets and Benchmarks*, 2023.

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[6] Yu Su, Shunyu Yao, Tao Yu, and Diyi Yang. Language agents: Foundations, prospects, and risks. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Tutorial Abstracts*, 2024.

[7] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[8] Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi.

How far can camels go? exploring the state of instruction tuning on open resources. *arXiv preprint arXiv:2306.04751*, 2023.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

A Appendix

A.1 Failure Analysis of Vanilla and Self-Play SFT Baselines

As shown in Table 1 , the self-play SFT model improves over the base model in several rounds, but both models still produce multiple 0v0 all-tie rounds caused by invalid submissions or service failures. Table 5 shows that the dominant failure modes include accidental removal of the startup block (e.g., `run_server`) and structural corruption (e.g., `return outside function` and broken `def move` signatures). A system-level safeguard reduces startup-block removal but does not eliminate instability, since aggressive structural edits such as large-range `sed` commands or full file overwrites can still introduce invalid code. Distillation from stronger frontier models (Claude-4/4.5) and finetuning on other agent trajectories (`qwen-coder-plus`, `gemini-2.5-pro`) show similar structural failure patterns.

Behaviorally, Qwen-based agents perform fewer structured diagnostic actions (Table 6 and Figure 2) and tend to apply edits without sufficient intermediate validation. Large-range modifications, such as full overwrites or truncate-and-append patterns, frequently disrupt code structure and increase the risk of syntax and runtime errors. This combination of limited diagnostic effort and high-risk editing strategies reduces robustness in tournament settings, particularly for Qwen3-30B.

These results suggest that improving tournament performance requires not only stronger supervision but also explicit behavior-level safeguards, such as pre-submit `py_compile` checks and startup-block validation, together with more systematic diagnostic procedures. Instruction tuning alone can shape response patterns, but it does not ensure runtime validity. This motivates our shift from response-level imitation toward learning action-level validity, so that edits remain executable and structurally safe within the deployment scaffold.

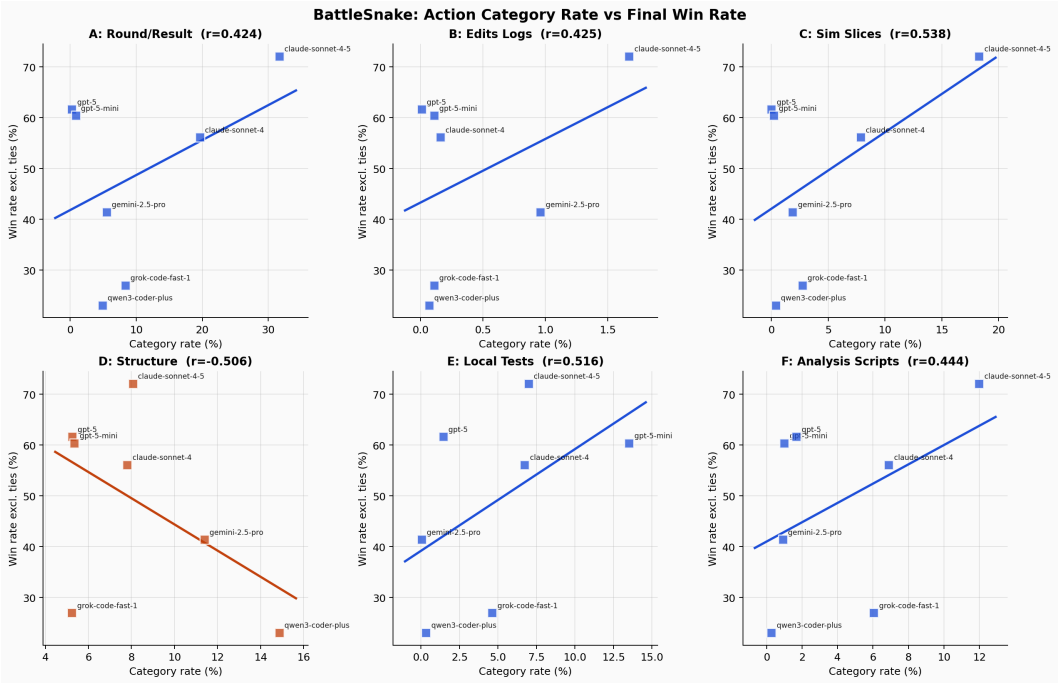

Figure 2: Correlation between error-checking and diagnosis behavior and win rate in the BattleSnake arena.

Table 5: Most frequent failure modes in Qwen self-play and the corresponding agent actions that triggered them, ordered by empirical frequency.

Observed Failure	**Typical Agent Action Trigger**
Startup entrypoint removed (`run_server` block missing)	Full-file overwrite of `main.py` (`cat > main.py`) or truncate-to-EOF edits (`sed -i 'x,$d' main.py`)
Syntax / structural corruption (e.g., `return outside function`)	Large-range deletion plus append replacement; partial function rewrite with indentation drift
Invalid metadata response (`invalid character '<'`)	File rewrite that compiles but causes the server to return malformed or non-JSON output

Table 6: Definitions, purposes, and training value of agent-side diagnostic action categories.

Action	**Representative Actions**	**Primary Purpose**	**Training / Evaluation Value**
A	`ls -la /logs/rounds/` `cat results.json`	Confirm win/loss outcomes and anomalous rounds	Builds the outcome-feedback loop and avoids blind edits
B	`ls -la /logs/edits/` `tail -n ...`	Inspect edit trajectories and submit status	Connects what changed with what happened
C	`cat sim_<id>.jsonl | tail -n 5`	Inspect failed or critical simulations	Localizes crash and timeout patterns for targeted fixes
D	`grep -n "def move" main.py`	Verify critical code structure	Prevents entrypoint or signature corruption
E	`pytest` `python test_*.py`	Validate logic before submission	Shifts failures left to local checks
F	`python analyze_round.py`	Aggregate multi-game patterns	Enables reusable cross-round analysis